

Patrick Glanville Support Tracker Process Guide

Chapter 1. Purpose and Scope

This project tracks urgent and non-urgent support tasks for Patrick Glanville and provides a repeatable process for creating and emailing the daily Urgency Report.

The project has two automation lanes:

- ? the main tracker dashboard in the project root
- ? the Email-folder automation flow for report generation and delivery

The 'Claude - Anthropic' folder is treated as an independent reference process. It contains working Gmail configuration that may be read by other scripts, but it is not the primary Email-folder automation path.

Chapter 2. Dashboard Workflow

The dashboard entry files are:

- ? 'index.html'
- ? 'app.js'
- ? 'styles.css'

Main dashboard actions:

- ? 'Save PDF To DB': upload shared PDF documents into the tracker database
- ? 'View Saved PDFs': open PDFs stored in the shared database
- ? 'Urgency Report': prepare the current Urgency Report workflow
- ? 'Create HTML Email': export the current urgency report as a rich HTML file
- ? 'History': review task updates
- ? 'Export' and 'Import': move tracker data in and out of JSON files

The dashboard is responsible for organizing tasks, calculating urgency content, and providing a manual operator workflow. It is not the scheduled sender by itself.

Chapter 3. Urgency Report Content

The Urgency Report is built from tasks marked with 'Urgent' priority.

The report emphasizes:

- ? due date
- ? task owner
- ? current percent complete
- ? next required action
- ? context notes

Urgent job and income tasks are grouped first because restoring income is treated as the highest-leverage path for transportation, housing, debt, and daily living stability.

Chapter 4. Email Folder Scripts

The 'Email' folder contains the main local automation scripts:

- ? 'Invoke-DailyUrgencyReport.ps1'
- ? 'Send-RichUrgencyReport.ps1'
- ? 'Watch-RichUrgencyReports.ps1'
- ? 'Set-UrgencyReportSmtplibCredential.ps1'
- ? 'send_rich_urgency_report.py'
- ? 'scheduled_urgency_report_sender.py'
- ? 'Start-DailyUrgencyEmailScheduler.ps1'

Roles of these scripts:

- ? 'Invoke-DailyUrgencyReport.ps1': pulls the current tracker state from Supabase and builds a fresh 'patrick-urgency-report-YYYY-MM-DD.html' file
- ? 'Send-RichUrgencyReport.ps1': sends or drafts the HTML report through Outlook or SMTP
- ? 'Watch-RichUrgencyReports.ps1': reacts when a new HTML report is dropped into the Email folder
- ? 'Set-UrgencyReportSmtplibCredential.ps1': stores encrypted SMTP credentials for the local machine
- ? 'send_rich_urgency_report.py': Python SMTP comparison utility for sending the latest saved HTML report
- ? 'scheduled_urgency_report_sender.py': dedicated daily scheduler for the Email-folder automation
- ? 'Start-DailyUrgencyEmailScheduler.ps1': PowerShell launcher for the dedicated daily scheduler

Chapter 5. Independent Claude Reference Process

The file:

? 'Claude - Anthropic/send_daily_email.py'

is a separate independent process.

It contains:

- ? the Gmail sender address
- ? the Gmail app password
- ? the recipient list
- ? an older standalone scheduling flow

This file is used as a reference source for Gmail settings by the separate Email-folder scheduler. The intended boundary is:

- ? it may be referenced
- ? it should not be modified as part of the Email-folder automation work unless explicitly requested

Chapter 6. Daily Automated 8:30 AM Process

The dedicated daily automation path is:

1. 'Start-DailyUrgencyEmailScheduler.ps1' launches 'scheduled_urgency_report_sender.py'
2. 'scheduled_urgency_report_sender.py' reads Gmail settings from 'Claude - Anthropic/send_daily_email.py'
3. At '8:30 AM', the scheduler runs 'Email/Invoke-DailyUrgencyReport.ps1 -GenerateOnly'
4. 'Invoke-DailyUrgencyReport.ps1' fetches the current tracker state from Supabase
5. A fresh urgency report HTML file is written into the 'Email' folder
6. The scheduler sends that report to all configured recipients using Gmail SMTP

This design separates:

- ? report generation
- ? email sending
- ? Gmail credential reference
- ? the independent Claude process

Chapter 7. Manual and Semi-Automatic Paths

Manual and semi-automatic alternatives remain available:

- ? run 'Invoke-DailyUrgencyReport.ps1 -GenerateOnly' to create the HTML without sending

- ? run 'Invoke-DailyUrgencyReport.ps1 -OpenDraft' to build the HTML and open a draft
- ? run 'Invoke-DailyUrgencyReport.ps1 -Send' to build the HTML and send immediately
- ? run 'Send-RichUrgencyReport.ps1' directly if a current HTML report already exists
- ? run 'Watch-RichUrgencyReports.ps1 -Send' if a watched folder flow is preferred

Chapter 8. Files and Data Dependencies

Important project dependencies include:

- ? 'supabase-config.js': contains Supabase URL and anon key reference for tracker data access
- ? the 'Email' folder: working location for generated HTML reports and send utilities
- ? 'smtp-settings.json' and 'smtp-credential.clixml': optional local SMTP credential storage
- ? saved PDF documents in the tracker database

If Supabase is unavailable, the automated report generation step cannot complete.

Chapter 9. Testing Procedure

Recommended test sequence:

1. Run 'Invoke-DailyUrgencyReport.ps1 -GenerateOnly'
2. Confirm a new 'patrick-urgency-report-YYYY-MM-DD.html' file appears in the 'Email' folder
3. Run 'Start-DailyUrgencyEmailScheduler.ps1'
4. Temporarily set the scheduler send time to one or two minutes ahead of the current time
5. Confirm the scheduler generates a fresh report and sends the email
6. Restore the scheduler send time to '08:30'

If the scheduler starts and prints its configuration, it is waiting normally for the target time.

Chapter 10. Recovery and Troubleshooting

Common issues:

- ? Python version incompatibility: older Python versions may reject newer type-hint syntax
- ? Supabase connection failure: report generation cannot complete if the tracker state

cannot be fetched

? SMTP failure: the Gmail sender, app password, or connection details may be wrong

? stale report problem: a sender that only mails the latest existing HTML file may not generate a fresh report first

Recovery approach:

1. confirm the scheduler starts cleanly
2. confirm 'Invoke-DailyUrgencyReport.ps1 -GenerateOnly' can build the report
3. confirm the Gmail sender and app password remain correct
4. confirm the separate Email-folder scheduler is the process being launched
5. confirm the 'Claude - Anthropic' process is being used only as a reference source when intended

Chapter 11. Script Reference Summary

Project root:

? 'index.html': dashboard structure and action buttons

? 'app.js': dashboard logic, urgency report rendering, and button behavior

? 'styles.css': dashboard styling

? 'PROCESS_GUIDE.md': human-readable source for this guide

Independent reference:

? 'Claude - Anthropic/send_daily_email.py': separate legacy scheduler with embedded Gmail settings

Email automation:

? 'Email/Invoke-DailyUrgencyReport.ps1': create daily urgency report HTML

? 'Email/Send-RichUrgencyReport.ps1': send or draft urgency report email

? 'Email/Watch-RichUrgencyReports.ps1': watcher-based send flow

? 'Email/send_rich_urgency_report.py': Python SMTP send helper

? 'Email/scheduled_urgency_report_sender.py': dedicated '8:30 AM' scheduler

? 'Email/Start-DailyUrgencyEmailScheduler.ps1': launcher for the dedicated scheduler